



**SWAMINARAYAN
UNIVERSITY**
THE RESERVOIR OF ALL KNOWLEDGE

**Faculty of Engineering (Degree)
Semester – 7th**

Subject Name – Information security

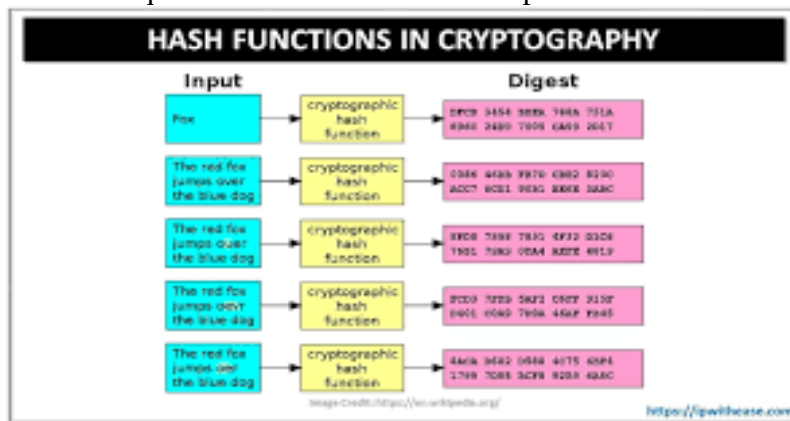
Subject Code – BE103707

Faculty Name – Radhika Gokani

CHAPTER -5

1. Cryptographic Hash Functions

A **cryptographic hash function** is a mathematical algorithm that takes an input (or 'message') and returns a fixed-size string of bytes, typically a digest that looks like a random sequence of characters. This output is called the **hash value** or **digest**.



Properties

To be considered cryptographically secure, a hash function must have the following properties:

1. **Deterministic:** The same input always produces the same output.
2. **Fast to compute:** The hash value should be quick to calculate for any given input.
3. **Preimage resistance (One-way):** Given a hash output h , it should be computationally infeasible to find any input x such that $\text{hash}(x) = h$.
4. **Second preimage resistance:** Given an input x_1 , it should be computationally infeasible to find another input x_2 ($x_2 \neq x_1$) such that $\text{hash}(x_1) = \text{hash}(x_2)$.
5. **Collision resistance:** It should be infeasible to find any two distinct inputs x_1 and x_2 such that $\text{hash}(x_1) = \text{hash}(x_2)$.
6. **Avalanche effect:** A small change in input should produce a significantly different hash output.

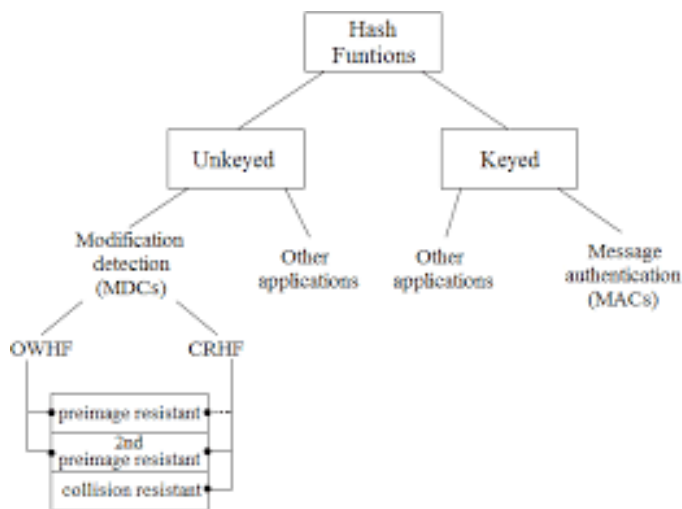
Common Uses

- **Data integrity verification:** Hashes verify that data has not been altered.
- **Password hashing:** Secure storage of passwords by hashing them.
- **Digital signatures:** Hashing a message before signing it.
- **Message authentication codes (MACs):** Hash-based message authentication.

- **Cryptocurrency mining:** Proof-of-work uses hash functions extensively.

Cryptographic Hash Functions

- **MD5** (Message Digest 5): 128-bit hash, now considered insecure due to collision vulnerabilities.
- **SHA-1** (Secure Hash Algorithm 1): 160-bit hash, also considered weak now.
- **SHA-2 family:** Includes SHA-224, SHA-256, SHA-384, SHA-512; widely used and currently secure.
- **SHA-3:** Latest standard based on Keccak, designed as a backup to SHA-2.



Example

Let's say you hash the word "hello" with SHA-256.

The output is:

2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824

If you change the input to "Hello" (capital H), the hash changes drastically:

185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969

2. Applications

Applications of Cryptographic Hash Functions

1. Data Integrity Verification

- Hashes are used to verify that data has not been altered during transmission or storage.
- Example: File downloads often provide a hash checksum so users can verify the file's integrity.

2. Password Storage

- Passwords are hashed before being stored in databases to protect against theft.
- Even if the database is leaked, attackers can't recover the original password easily.
- Often combined with **salts** to prevent precomputed attacks (rainbow tables).

3. Digital Signatures

- Instead of signing an entire document, a hash of the document is signed.
- This improves efficiency and security by signing a fixed-size digest instead of large data.

4. Message Authentication Codes (MACs)

- Used to verify the authenticity and integrity of a message.

- Typically involves a secret key and a hash function (e.g., HMAC).
5. **Blockchain and Cryptocurrencies**
 - Cryptographic hashes are fundamental to blockchain technology.
 - They ensure immutability of data and link blocks together securely.
 - Mining involves finding a hash that meets a certain difficulty target.
 6. **Commitment Schemes**
 - Hashes are used to commit to a value while keeping it hidden until later.
 - Useful in protocols like zero-knowledge proofs and secure voting.
 7. **Random Number Generation**
 - Hash functions can be part of algorithms that produce cryptographically secure random numbers.
 8. **Fingerprinting and Data Deduplication**
 - Hashes help detect duplicate files by comparing hash values rather than entire file content.
 9. **Secure Key Derivation Functions**
 - Hash functions are used in algorithms like PBKDF2 or HKDF to derive keys from passwords or other keying material.

Summary Table

Application	Purpose
Data Integrity	Verify data hasn't been altered
Password Storage	Securely store passwords
Digital Signatures	Efficient signing of messages
Message Authentication	Authenticate messages and verify integrity
Blockchain & Cryptocurrencies	Secure blockchain data and mining
Commitment Schemes	Commit to hidden values
Random Number Generation	Generate secure random numbers
Fingerprinting/Deduplication	Identify duplicates efficiently
Key Derivation Functions	Generate cryptographic keys

3. Simple hash functions

Simple hash functions are algorithms that convert input data into a fixed-size hash value, usually for purposes like:

- Data indexing
- Hash tables (e.g., in programming)
- Basic checksums

They are **not designed for security** and **lack strong cryptographic properties** like collision resistance or preimage resistance.

Characteristics of Simple Hash Functions

- **Fast and lightweight**
- May produce collisions more frequently
- No guarantee against intentional manipulation
- Often used in data structures and checksums

Examples of Simple Hash Functions

1. Division Method

- Hash value = key mod table_size
- Common in hash tables but easy to predict.

2. Sum of Characters

- Add ASCII values of all characters in a string.
- For example, "cat" $\rightarrow 99 + 97 + 116 = 312$.

3. DJB2 Hash Function (Simple, Popular)

unsigned long hash = 5381;

int c;

```
while (c = *str++) {
```

```
    hash = ((hash << 5) + hash) + c; // hash * 33 + c
```

```
}
```

- Fast and simple, often used in practice for hash tables.
- Not cryptographically secure.

4. Polynomial Rolling Hash

- Uses polynomial accumulation of characters, often for string matching.

When to Use Simple Hash Functions?

- Building hash tables for efficient data lookup.
- Basic error detection (simple checksums).
- Non-security-critical data indexing.

When Not to Use?

- Any security-related purpose.
- Password storage.
- Digital signatures or data integrity where adversaries may try to tamper.

4. Its requirements and security

Requirements of Hash Functions

A good hash function—whether simple or cryptographic—should meet these basic criteria:

- **Deterministic:** The same input always produces the same output.
- **Fixed Output Size:** Regardless of input size, the output (hash value) is of fixed length.
- **Efficient to Compute:** It should be fast and easy to compute the hash value for any input.
- **One-Way Function:** It should be computationally infeasible to reverse the hash and retrieve the original input.
- **Collision Resistance:**
 - **Weak Collision Resistance:** Hard to find two different inputs that produce the same hash.
 - **Strong Collision Resistance:** Hard to find *any* two inputs that hash to the same value.
- **Avalanche Effect:** A small change in input should produce a significantly different hash output.

Security Considerations

Simple hash functions like XOR-based ones are easy to implement but **not secure** for cryptographic purposes. Here's why:

- **Susceptible to Collisions:** Simple hashes often produce the same output for different inputs, especially with structured data.
- **Predictable Output:** Without sufficient randomness or complexity, attackers can reverse-engineer or forge inputs.
- **No Resistance to Tampering:** If used in authentication or integrity checks, attackers can manipulate data and recreate valid hashes.

For secure applications like password storage, digital signatures, or blockchain, **cryptographic hash functions** like SHA-256 or SHA-3 are preferred. These are designed to resist brute-force attacks and ensure data authenticity

5. Hash functions based on Cipher Block Chaining

Hash functions based on Cipher Block Chaining (CBC) are an intriguing class of cryptographic constructions that leverage block ciphers—like DES or AES—without necessarily using a secret key.

CBC-Based Hashing?

CBC-based hash functions mimic the Cipher Block Chaining mode used in encryption, but instead of encrypting data, they process it to produce a fixed-length hash value.

Basic Steps:

1. **Message Division:** The input message is divided into fixed-size blocks: $M_1, M_2, \dots, M_{N-1}, M_N$.
2. **Initialization:** Start with an initial value (often zero or a predefined constant).
3. **Chaining:** Each block is processed using a block cipher function E , chaining the output with the next block:
4. **Final Output:** The last output H_N becomes the hash value.

Importantly, **no secret key is used**—the block cipher operates in a fixed mode, often with a constant key or as a permutation.

Vulnerabilities & Attacks

Despite their elegance, CBC-based hash functions have notable weaknesses:

- **Birthday Attacks:** If the hash output is small (e.g., 64 bits), collisions can be found with 2^{32} effort.
- **Meet-in-the-Middle Attacks:** Attackers can craft messages that produce the same hash as a legitimate one by exploiting the chaining structure.
- **Lack of Keying:** Without a secret key, the hash function is more predictable and easier to manipulate.

Modern Alternatives

Due to these limitations, modern cryptography favors dedicated hash functions like:

Hash Function	Strengths	Common Use
SHA-2 (e.g., SHA-256)	Strong security, widely supported	Digital signatures, blockchain
SHA-3	Based on Keccak sponge construction	Post-quantum resilience
BLAKE2	Fast and secure	General-purpose hashing

6. Secure Hash Algorithm (SHA)

- **Secure Hash Algorithm (SHA)** family is a cornerstone of modern cryptography, designed to ensure data integrity, authentication, and security across digital systems.
- SHA stands for **Secure Hash Algorithm**, a group of cryptographic hash functions developed by the **National Security Agency (NSA)** and standardized by **NIST**.
- These algorithms take an input (or "message") and produce a fixed-size string of bytes—typically a **digest** that uniquely represents the input.

Properties:

- **Deterministic:** Same input always yields the same output.
- **Irreversible:** You can't reconstruct the original input from the hash.

- **Collision-resistant:** Hard to find two inputs that produce the same hash (except SHA-1, which is now vulnerable).
- **Avalanche effect:** Small input changes cause large output changes.

Applications of SHA

SHA algorithms are used in:

- **Digital signatures**
- **SSL/TLS certificates**
- **Blockchain and cryptocurrency**
- **Password hashing**
- **Data integrity verification**

Comparison of SHA-1, SHA-2, and SHA-3

Feature	SHA-1	SHA-2	SHA-3
Release Year	1995	2001	2015
Digest Size	160 bits	224, 256, 384, 512 bits	224, 256, 384, 512 bits
Structure	Merkle–Damgård	Merkle–Damgård with improved padding & logic	Sponge construction (Keccak)
Security Status	Broken (collision attacks feasible)	Secure (widely used, especially SHA-256)	Secure (alternative to SHA-2)
Performance	Fast but insecure	Moderate speed, good security	Flexible, slower in hardware, fast in software
Collision Resistance	Weak (collisions found)	Strong	Strong
Use Cases	Legacy systems (deprecated)	Digital signatures, SSL/TLS, blockchain	Post-quantum cryptography, modern apps
Standardization	FIPS PUB 180-1	FIPS PUB 180-2/4	FIPS PUB 202

SHA Family Overview

Version	Output Size	Status	Notes
SHA-0	160 bits	Obsolete	Withdrawn due to flaws
SHA-1	160 bits	Deprecated	Vulnerable to collision attacks
SHA-2	224–512 bits	Widely used	Includes SHA-224, SHA-256, SHA-384, SHA-512
SHA-3	224–512 bits	Latest standard	Based on Keccak sponge construction
